

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP THỰC HÀNH 2

Họ và tên: **Bùi Đăng Dương**
MSV: **23020795**

Mục lục

1	Môi trường thực nghiệm	3
2	Bài 1 — Module hello	3
2.1	Mục tiêu	3
2.2	Mã nguồn (b1/hello.c)	3
2.3	Các bước thực hiện	3
2.4	Kết quả	4
2.5	Nhận xét	4
3	Bài 2 — Module hello_param (tham số module)	4
3.1	Mục tiêu	4
3.2	Mã nguồn (b2/hello_param.c)	4
3.3	Các bước thực hiện	5
3.4	Kết quả	5
3.5	Nhận xét	5
4	Bài 3 — In dãy số và tính tổng bình phương	6
4.1	Mục tiêu	6
4.2	Mã nguồn (b3/b3.c)	6
4.3	Các bước thực hiện	7
4.4	Kết quả	7
4.5	Nhận xét	7
5	Bài 4 — Module hello_array (tham số mảng)	7
5.1	Mục tiêu	7
5.2	Mã nguồn (b4/hello_array.c)	7
5.3	Các bước thực hiện	8
5.4	Kết quả	9
5.5	Nhận xét	9
6	Bài 5 — Module hello_multi (nhiều file nguồn)	9
6.1	Mục tiêu	9
6.2	Mã nguồn	9

6.3	Các bước thực hiện	10
6.4	Kết quả	10
6.5	Nhận xét	11
7	Bài 6 — Module <code>hello_proc</code> (thông tin tiến trình)	11
7.1	Mục tiêu	11
7.2	Mã nguồn (<code>b6/hello_proc.c</code>)	11
7.3	Các bước thực hiện	11
7.4	Kết quả	12
7.5	Nhận xét	12
8	Bài 7 — Module <code>hello_sysfs</code> (tham số <code>sysfs</code>)	12
8.1	Mục tiêu	12
8.2	Mã nguồn (<code>b7/hello_sysfs.c</code>)	12
8.3	Các bước thực hiện	13
8.4	Kết quả	13
8.5	Nhận xét	13
9	Bài 8 — Provider / Consumer (<code>EXPORT_SYMBOL</code>)	13
9.1	Mục tiêu	13
9.2	Mã nguồn	14
9.3	Các bước thực hiện	16
9.4	Kết quả	16
9.5	Nhận xét	16
10	Kết luận	16

1 Môi trường thực nghiệm

- Nhân Linux 5.15.163 tự biên dịch (LOCALVERSION=-edu), chạy trên máy ảo QEMU (qemu-system-x86_64).
- Hệ điều hành khách: **Ubuntu 24.04 (noble)**, được cài đặt bằng debootstrap lên ảnh đĩa rootfs.img (định dạng ext4).
- Biên dịch module bằng hệ thống kbuild với KDIR trỏ tới cây nguồn linux-5.15.163.
- Nạp module bằng insmod, gỡ module bằng rmmod, quan sát nhật ký nhân bằng dmesg.

2 Bài 1 — Module hello

2.1 Mục tiêu

Xây dựng module nhân đơn giản nhất: khi nạp module, nhân in ra thông điệp chào; khi gỡ module, nhân in ra thông điệp tạm biệt.

2.2 Mã nguồn (b1/hello.c)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("Your Name");
7 MODULE_DESCRIPTION("Hello Kernel");
8
9 static int __init hello_init(void) {
10     printk(KERN_INFO "Hello kernel!\n");
11     return 0;
12 }
13
14 static void __exit hello_exit(void) {
15     printk(KERN_INFO "Goodbye kernel!\n");
16 }
17
18 module_init(hello_init);
19 module_exit(hello_exit);
```

2.3 Các bước thực hiện

```
sudo insmod /root/lkm/hello.ko
dmesg | tail -3
sudo rmmod hello
```

```
dmesg | tail -3
```

2.4 Kết quả

```
student@uetsys:~ insmod /root/lkm/hello.ko
student@uetsys:~ dmesg
[ 16.294104] hello: loading out-of-tree module taints kernel.
[ 16.298816] Hello kernel!
student@uetsys:~ rmmod hello
student@uetsys:~ dmesg
[ 16.294104] hello: loading out-of-tree module taints kernel.
[ 16.298816] Hello kernel!
[ 16.826392] Goodbye kernel!
```

2.5 Nhận xét

Hai macro `module_init` và `module_exit` đăng ký hàm khởi tạo và hàm kết thúc của module với nhân. Hàm khởi tạo được gọi trong ngữ cảnh của tiến trình `insmod`, trả về 0 báo nạp thành công. `printk` với mức `KERN_INFO` đưa thông điệp vào bộ đệm log của nhân, xem được bằng `dmesg`. Dòng `taints kernel` là cảnh báo thông thường khi nạp module ngoài cây nguồn chính thống, không ảnh hưởng tới chức năng. Khai báo `MODULE_LICENSE("GPL")` là bắt buộc để tránh nhân bị đánh dấu *tainted* do giấy phép.

3 Bài 2 — Module `hello_param` (tham số module)

3.1 Mục tiêu

Module nhận hai tham số dòng lệnh: `whom` (chuỗi ký tự, mặc định "world") và `howmany` (số nguyên, mặc định 1). Khi nạp, module in lời chào đúng `howmany` lần; khi gỡ, in lời tạm biệt.

3.2 Mã nguồn (`b2/hello_param.c`)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("Your Name");
7 MODULE_DESCRIPTION("Hello LKM with parameters");
8
9 static char *whom = "world";
10 module_param(whom, charp, 0644);
11 MODULE_PARM_DESC(whom, "Whom to greet (default: world)");
12
13 static int howmany = 1;
14 module_param(howmany, int, 0644);
```

```

15 MODULE_PARAM_DESC(howmany, "Number of greetings (default: 1)");
16
17 static int __init hello_param_init(void)
18 {
19     int i;
20
21     if (howmany < 1) {
22         printk(KERN_INFO "hello_param: howmany must be >= 1, using 1\n");
23         howmany = 1;
24     }
25
26     for (i = 1; i <= howmany; i++) {
27         printk(KERN_INFO "hello_param: Hello, %s! [%d/%d]\n",
28             whom, i, howmany);
29     }
30     return 0;
31 }
32
33 static void __exit hello_param_exit(void)
34 {
35     printk(KERN_INFO "hello_param: Bye, %s!\n", whom);
36 }
37
38 module_init(hello_param_init);
39 module_exit(hello_param_exit);

```

3.3 Các bước thực hiện

```

sudo insmod /root/lkm/hello_param.ko whom=UET howmany=3
dmesg | tail -5
sudo rmmod hello_param
dmesg | tail -2

```

3.4 Kết quả

```

student@uetsys:~$ insmod /root/lkm/hello_param.ko whom=UET howmany=3
student@uetsys:~$ dmesg
[ 17.630032] hello_param: Hello, UET! [1/3]
[ 17.630087] hello_param: Hello, UET! [2/3]
[ 17.630106] hello_param: Hello, UET! [3/3]
student@uetsys:~$ rmmod hello_param
student@uetsys:~$ dmesg
[ 17.630106] hello_param: Hello, UET! [3/3]
[ 18.169426] hello_param: Bye, UET!

```

3.5 Nhận xét

Macro `module_param(tên, kiểu, quyền)` biến một biến toàn cục thành tham số có thể truyền khi `insmod`. Kiểu `charp` dùng cho chuỗi, `int` dùng cho số nguyên; quyền

0644 cho phép đọc tham số qua sysfs. Chương trình có kiểm tra biên (`howmany < 1`) để tránh vòng lặp rỗng hoặc hành vi không xác định khi người dùng truyền giá trị không hợp lệ.

4 Bài 3 — In dãy số và tính tổng bình phương

4.1 Mục tiêu

Module nhận tham số `n` (mặc định 1). Khi nạp: in các số nguyên từ 1 đến `n` và in tổng bình phương của chúng. Khi gỡ: in thông điệp Done with `n=<giá trị>`.

4.2 Mã nguồn (b3/b3.c)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("cabylock");
7 MODULE_DESCRIPTION("Print 1..n and sum of squares");
8
9 static int n = 1;
10 module_param(n, int, 0644);
11 MODULE_PARM_DESC(n, "Upper bound (default: 1)");
12
13 static int __init ini_mod(void)
14 {
15     int i;
16     long long s = 0;
17
18     if (n < 1) {
19         printk(KERN_INFO "b3: n must be >= 1, using 1\n");
20         n = 1;
21     }
22
23     for (i = 1; i <= n; i++) {
24         printk(KERN_INFO "b3: i = %d\n", i);
25         s += (long long)i * i;
26     }
27
28     printk(KERN_INFO "b3: sum of squares 1..%d = %lld\n", n, s);
29     return 0;
30 }
31
32 static void __exit exit_mod(void)
33 {
34     printk(KERN_INFO "Done with n=%d\n", n);
35 }
36
```

```
37 module_init(ini_mod);
38 module_exit(exit_mod);
```

4.3 Các bước thực hiện

```
sudo insmod /root/lkm/b3.ko n=5
dmesg | tail -8
sudo rmmod b3
dmesg | tail -2
```

4.4 Kết quả

```
student@uetsys:~$ insmod /root/lkm/b3.ko n=5
student@uetsys:~$ dmesg
[ 18.930151] b3: i = 1
[ 18.930212] b3: i = 2
[ 18.930220] b3: i = 3
[ 18.930223] b3: i = 4
[ 18.930227] b3: i = 5
[ 18.930234] b3: sum of squares 1..5 = 55
student@uetsys:~$ rmmod b3
student@uetsys:~$ dmesg
[ 18.930234] b3: sum of squares 1..5 = 55
[ 19.502423] Done with n=5
```

4.5 Nhận xét

Với $n = 5$, log ghi nhận đầy đủ dãy $i = 1..5$ và tổng bình phương $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$, trùng với công thức $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$. Biến tích lũy dùng kiểu `long long` (in bằng `%lld`) để tránh tràn số khi n lớn. Khi gỡ, module đọc lại giá trị hiện tại của n để in ra, cho thấy tham số module tồn tại suốt vòng đời của module.

5 Bài 4 — Module `hello_array` (tham số mảng)

5.1 Mục tiêu

Module nhận tham số mảng số nguyên `nums` (tối đa 10 phần tử). Khi nạp, in từng phần tử và tổng của mảng. Khi gỡ, in thông điệp `hello_array: unloaded`.

5.2 Mã nguồn (`b4/hello_array.c`)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("cabylock");
7 MODULE_DESCRIPTION("Hello LKM with array parameter");
```

```

8
9 #define MAX_NUMS 10
10
11 static int nums[MAX_NUMS];
12 static int nums_count = 1;
13 static int nums_argc = 0;
14
15 module_param_array(nums, int, &nums_argc, 0644);
16 MODULE_PARM_DESC(nums, "Array of integers (max 10 elements)");
17
18 static int __init hello_array_init(void)
19 {
20     int i;
21     long long sum = 0;
22
23     if (nums_argc > MAX_NUMS) {
24         printk(KERN_INFO "hello_array: too many elements (%d), truncating to %d\n"
25             ,
26             nums_argc, MAX_NUMS);
27         nums_count = MAX_NUMS;
28     } else if (nums_argc < 1) {
29         printk(KERN_INFO "hello_array: no elements given, using default {0}\n");
30         nums[0] = 0;
31         nums_count = 1;
32     } else {
33         nums_count = nums_argc;
34     }
35
36     for (i = 0; i < nums_count; i++) {
37         printk(KERN_INFO "hello_array: nums[%d] = %d\n", i, nums[i]);
38         sum += nums[i];
39     }
40
41     printk(KERN_INFO "hello_array: sum = %lld\n", sum);
42     return 0;
43 }
44
45 static void __exit hello_array_exit(void)
46 {
47     printk(KERN_INFO "hello_array: unloaded\n");
48 }
49
50 module_init(hello_array_init);
51 module_exit(hello_array_exit);

```

5.3 Các bước thực hiện

```

sudo insmod /root/lkm/hello_array.ko nums=1,2,3,4,5
dmesg | tail -8
sudo rmmod hello_array

```

```
dmesg | tail -2
```

5.4 Kết quả

```
student@uetsys:~ insmod /root/lkm/hello_array.ko nums=1,2,3,4,5
student@uetsys:~ dmesg
[ 20.303870] hello_array: nums[0] = 1
[ 20.303932] hello_array: nums[1] = 2
[ 20.303939] hello_array: nums[2] = 3
[ 20.303943] hello_array: nums[3] = 4
[ 20.303947] hello_array: nums[4] = 5
[ 20.303956] hello_array: sum = 15
student@uetsys:~ rmmod hello_array
student@uetsys:~ dmesg
[ 20.303956] hello_array: sum = 15
[ 20.853338] hello_array: unloaded
```

5.5 Nhận xét

Macro `module_param_array` nhận danh sách giá trị cách nhau bằng dấu phẩy trên dòng lệnh `insmod`; con trỏ `&nums_argc` nhận số phần tử thực tế người dùng truyền vào. Chương trình xử lý đầy đủ ba trường hợp: quá 10 phần tử (cắt bớt), không truyền gì (dùng mặc định `{0}`), và trường hợp thông thường. Với đầu vào 1, 2, 3, 4, 5, tổng ghi nhận là 15, chính xác.

6 Bài 5 — Module `hello_multi` (nhiều file nguồn)

6.1 Mục tiêu

Xây dựng module gồm hai file nguồn: `main.c` chứa hàm khởi tạo/kết thúc và gọi hàm từ file phụ; `helper.c` định nghĩa hàm `helper_greet()` in log. Khi nạp, module in thông điệp loaded rồi gọi hàm phụ trợ; khi gỡ, in thông điệp unloaded.

6.2 Mã nguồn

```
1 #ifndef HELPER_H
2 #define HELPER_H
3
4 void helper_greet(void);
5
6 #endif
```

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3
4 void helper_greet(void)
5 {
6     printk(KERN_INFO "hello_multi: greeting from helper_greet()\n");
7 }
```

```

1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include "helper.h"
5
6 MODULE_LICENSE("GPL");
7 MODULE_AUTHOR("cabylock");
8 MODULE_DESCRIPTION("Hello LKM split into two source files");
9
10 static int __init hello_multi_init(void)
11 {
12     printk(KERN_INFO "hello_multi: module loaded\n");
13     helper_greet();
14     return 0;
15 }
16
17 static void __exit hello_multi_exit(void)
18 {
19     printk(KERN_INFO "hello_multi: module unloaded\n");
20 }
21
22 module_init(hello_multi_init);
23 module_exit(hello_multi_exit);

```

File Makefile khai báo liên kết hai đối tượng thành một module:

```

obj-m += hello_multi.o
hello_multi-objs := main.o helper.o

```

6.3 Các bước thực hiện

```

sudo insmod /root/lkm/hello_multi.ko
dmesg | tail -3
sudo rmmod hello_multi
dmesg | tail -2

```

6.4 Kết quả

```

student@uetsys:~$ insmod /root/lkm/hello_multi.ko
student@uetsys:~$ dmesg
[ 21.647305] hello_multi: module loaded
[ 21.647363] hello_multi: greeting from helper_greet()
student@uetsys:~$ rmmod hello_multi
student@uetsys:~$ dmesg
[ 21.647363] hello_multi: greeting from helper_greet()
[ 22.174043] hello_multi: module unloaded

```

6.5 Nhận xét

Bài này minh họa cách tổ chức module gồm nhiều file nguồn: kbuild biên dịch riêng từng file .c thành .o rồi liên kết tĩnh thành một file .ko duy nhất theo biến <tên>-objs. Hàm helper_greet() là liên kết tĩnh nội bộ của module (không xuất ra ngoài), khác với cơ chế EXPORT_SYMBOL dùng chung giữa các module ở Bài 8. Log cho thấy đúng thứ tự: thông điệp loaded trước, lời chào từ hàm phụ trợ sau.

7 Bài 6 — Module hello_proc (thông tin tiến trình)

7.1 Mục tiêu

Khi nạp module, in tên tiến trình gọi nạp (current->comm) và mã định danh tiến trình (current->pid). Khi gỡ, in thông điệp unloaded.

7.2 Mã nguồn (b6/hello_proc.c)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/sched.h>
5
6 MODULE_LICENSE("GPL");
7 MODULE_AUTHOR("cabylock");
8 MODULE_DESCRIPTION("Print current process name and PID");
9
10 static int __init hello_proc_init(void)
11 {
12     printk(KERN_INFO "hello_proc: loaded by process \"%s\" (pid=%d)\n",
13            current->comm, current->pid);
14     return 0;
15 }
16
17 static void __exit hello_proc_exit(void)
18 {
19     printk(KERN_INFO "hello_proc: unloaded\n");
20 }
21
22 module_init(hello_proc_init);
23 module_exit(hello_proc_exit);
```

7.3 Các bước thực hiện

```
sudo insmod /root/lkm/hello_proc.ko
dmesg | tail -2
sudo rmmod hello_proc
dmesg | tail -2
```

7.4 Kết quả

```
student@uetsys:~ insmod /root/lkm/hello_proc.ko
student@uetsys:~ dmesg
[ 22.951299] hello_proc: loaded by process "insmod" (pid=340)
student@uetsys:~ rmmod hello_proc
student@uetsys:~ dmesg
[ 22.951299] hello_proc: loaded by process "insmod" (pid=340)
[ 23.460261] hello_proc: unloaded
```

7.5 Nhận xét

Hàm khởi tạo của module chạy trong ngữ cảnh của tiến trình người dùng gọi `insmod`, do đó con trỏ `current` (kiểu `struct task_struct`, khai báo trong `linux/sched.h`) chính là tiến trình `insmod` đó. Kết quả thực nghiệm ghi nhận đúng: loaded by process "insmod". Trường `comm` (tối đa 16 ký tự) là tên rút gọn của tiến trình. Bài này cho thấy module nhân có thể quan sát trực tiếp trạng thái tiến trình đang chạy.

8 Bài 7 — Module `hello_sysfs` (tham số `sysfs`)

8.1 Mục tiêu

Module có tham số `val` (mặc định 0) được khai báo với quyền 0644 để cho phép đọc/ghi ngay cả khi module đang chạy, thông qua hệ thống file `sysfs`.

8.2 Mã nguồn (`b7/hello_sysfs.c`)

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("cabylock");
7 MODULE_DESCRIPTION("Hello LKM with sysfs-readable parameter");
8
9 static int val = 0;
10 module_param(val, int, 0644);
11 MODULE_PARM_DESC(val, "An integer value (default: 0)");
12
13 static int __init hello_sysfs_init(void)
14 {
15     printk(KERN_INFO "hello_sysfs: loaded with val=%d\n", val);
16     return 0;
17 }
18
19 static void __exit hello_sysfs_exit(void)
20 {
21     printk(KERN_INFO "hello_sysfs: unloaded\n");
22 }
```

```
23  
24 module_init(hello_sysfs_init);  
25 module_exit(hello_sysfs_exit);
```

8.3 Các bước thực hiện

```
sudo insmod /root/lkm/hello_sysfs.ko val=10  
dmesg | tail -2  
cat /sys/module/hello_sysfs/parameters/val  
echo 42 | sudo tee /sys/module/hello_sysfs/parameters/val  
cat /sys/module/hello_sysfs/parameters/val  
sudo rmmod hello_sysfs  
dmesg | tail -2
```

8.4 Kết quả

```
student@uetsys:~ insmod /root/lkm/hello_sysfs.ko val=10  
student@uetsys:~ dmesg  
[ 24.247179] hello_sysfs: loaded with val=10  
student@uetsys:~ cat /sys/module/hello_sysfs/parameters/val  
10  
student@uetsys:~ echo 42 | sudo tee /sys/module/hello_sysfs/parameters/val  
42  
student@uetsys:~ cat /sys/module/hello_sysfs/parameters/val  
42  
student@uetsys:~ rmmod hello_sysfs  
student@uetsys:~ dmesg  
[ 24.247179] hello_sysfs: loaded with val=10  
[ 25.469494] hello_sysfs: unloaded
```

8.5 Nhận xét

Tham số thứ ba perm của module_param quyết định việc tạo file sysfs tương ứng tại /sys/module/<tên>/parameters/: 0 nghĩa là ẩn, 0644 nghĩa là cho phép đọc và cho phép root ghi trực tiếp. Thực nghiệm cho thấy: sau khi nạp với val=10, đọc file được 10; ghi 42 vào file rồi đọc lại được 42 mà không cần nạp lại module. Đây là cơ chế cấu hình động module trong lúc hệ thống đang vận hành.

9 Bài 8 — Provider / Consumer (EXPORT_SYMBOL)

9.1 Mục tiêu

Module Provider cung cấp hàm compute_stats tính tổng (sum), trung bình (avg), cực đại (max), cực tiểu (min) của một mảng số nguyên, trả kết quả qua struct stats_result, có kiểm tra tham số đầu vào. Module Consumer dùng hàm đã xuất (export) này trên mảng mẫu {10, 20, 30, 40, 50} và in kết quả ra log nhân.

9.2 Mã nguồn

```
1 #ifndef STATS_H
2 #define STATS_H
3
4 struct stats_result {
5     long long sum;
6     long long avg;
7     int max;
8     int min;
9 };
10
11 int compute_stats(const int *arr, int count, struct stats_result *out);
12
13 #endif
```

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/errno.h>
5 #include "stats.h"
6
7 MODULE_LICENSE("GPL");
8 MODULE_AUTHOR("cabylock");
9 MODULE_DESCRIPTION("Provider: exports compute_stats()");
10
11 int compute_stats(const int *arr, int count, struct stats_result *out)
12 {
13     int i;
14     long long sum = 0;
15
16     if (!arr || count <= 0 || !out)
17         return -EINVAL;
18
19     out->min = arr[0];
20     out->max = arr[0];
21
22     for (i = 0; i < count; i++) {
23         sum += arr[i];
24         if (arr[i] < out->min)
25             out->min = arr[i];
26         if (arr[i] > out->max)
27             out->max = arr[i];
28     }
29
30     out->sum = sum;
31     out->avg = sum / count;
32     return 0;
33 }
34 EXPORT_SYMBOL(compute_stats);
35
36 static int __init provider_init(void)
```

```

37 {
38     printk(KERN_INFO "provider: loaded (exports compute_stats)\n");
39     return 0;
40 }
41
42 static void __exit provider_exit(void)
43 {
44     printk(KERN_INFO "provider: unloaded\n");
45 }
46
47 module_init(provider_init);
48 module_exit(provider_exit);

```

```

1  #include <linux/module.h>
2  #include <linux/init.h>
3  #include <linux/kernel.h>
4  #include "stats.h"
5
6  MODULE_LICENSE("GPL");
7  MODULE_AUTHOR("cabylock");
8  MODULE_DESCRIPTION("Consumer: uses compute_stats() from provider");
9
10 static int __init consumer_init(void)
11 {
12     int data[] = { 10, 20, 30, 40, 50 };
13     int n = sizeof(data) / sizeof(data[0]);
14     struct stats_result r;
15     int ret;
16
17     ret = compute_stats(data, n, &r);
18     if (ret != 0) {
19         printk(KERN_ERR "consumer: compute_stats failed (%d)\n", ret);
20         return ret;
21     }
22
23     printk(KERN_INFO "consumer: input = {10, 20, 30, 40, 50}\n");
24     printk(KERN_INFO "consumer: sum = %lld\n", r.sum);
25     printk(KERN_INFO "consumer: avg = %lld\n", r.avg);
26     printk(KERN_INFO "consumer: max = %d\n", r.max);
27     printk(KERN_INFO "consumer: min = %d\n", r.min);
28     return 0;
29 }
30
31 static void __exit consumer_exit(void)
32 {
33     printk(KERN_INFO "consumer: unloaded\n");
34 }
35
36 module_init(consumer_init);
37 module_exit(consumer_exit);

```

9.3 Các bước thực hiện

```
sudo insmod /root/lkm/provider.ko
sudo insmod /root/lkm/consumer.ko
dmesg | tail -8
sudo rmmod consumer
sudo rmmod provider
dmesg | tail -3
```

9.4 Kết quả

```
student@uetsys:~ insmod /root/lkm/provider.ko
student@uetsys:~ insmod /root/lkm/consumer.ko
student@uetsys:~ dmesg
[ 26.314767] provider: loaded (exports compute_stats)
[ 26.579293] consumer: input = {10, 20, 30, 40, 50}
[ 26.579339] consumer: sum = 150
[ 26.579371] consumer: avg = 30
[ 26.579380] consumer: max = 50
[ 26.579388] consumer: min = 10
student@uetsys:~ rmmod consumer
student@uetsys:~ rmmod provider
student@uetsys:~ dmesg
[ 26.579388] consumer: min = 10
[ 27.086475] consumer: unloaded
[ 27.323107] provider: unloaded
```

9.5 Nhận xét

Macro `EXPORT_SYMBOL` đưa `compute_stats` vào bảng ký hiệu chung của nhân, cho phép module khác liên kết động tới (thể hiện ở trường `depends: provider` trong module `Consumer`). Thứ tự thao tác là bắt buộc: phải nạp `Provider` trước `Consumer`, và gỡ `Consumer` trước `Provider`; làm ngược lại nhân báo lỗi `Unknown symbol`. Hàm kiểm tra tính hợp lệ của đầu vào (con trỏ `NULL`, `count <= 0`) và trả về `-EINVAL` khi sai, tránh lỗi dereference gây oops/panic nhân. Với mảng mẫu, kết quả thực nghiệm là `sum=150`, `avg=30`, `max=50`, `min=10`, hoàn toàn chính xác.

10 Kết luận

Cả tám bài thực hành đều được biên dịch thành công cho nhân 5.15.163 và kiểm chứng thực tế trên máy ảo QEMU. Nội dung đi từ cơ chế nạp/gỡ cơ bản (Bài 1), tham số hóa module (Bài 2–4), tổ chức module nhiều file nguồn (Bài 5), truy cập thông tin tiến trình `current` (Bài 6), cấu hình động qua `sysfs` (Bài 7), đến chia sẻ hàm giữa các module bằng `EXPORT_SYMBOL` (Bài 8). Các kết quả trên `dmesg` đều khớp với mã nguồn và tính toán lý thuyết, cho thấy sinh viên đã nắm vững vòng đời và cơ chế giao tiếp của Loadable Kernel Module trong Linux.